The AI Operating System, Part IV:

Planner Engine — Order Book Dynamics and Natural Transformation

Matthew Long

The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yonedaai.com · https://yonedaai.com

March 10, 2026

Abstract

We present a categorical framework for the *planner engine* of an AI operating system, establishing it as the highest-level orchestration primitive analogous to `systemd`/`init` in classical operating systems. The planner is formalized as a *natural transformation* $\eta : F \Rightarrow G$ between the *task functor* F, mapping jobs to requirement specifications, and the *capability functor* G, mapping agents to offered capabilities. We introduce an *order book* structure modeled as a profunctor $P : \mathcal{A}^{\mathrm{op}} \times \mathcal{T} \to \mathbf{Set}$, through which agents (sellers) and clients (buyers) interact via a bidding protocol. Market clearing is characterized as a colimit in the transaction category, and the escrow mechanism is shown to satisfy the monad laws, guaranteeing financial consistency of credit operations. Job decomposition into directed acyclic graphs with topological execution ordering is proven to be functorial, preserving job semantics under decomposition. We validate the framework through an Elixir/OTP implementation featuring GenServer-based order books, Mnesia-backed escrow transactions, and supervision-tree-based task decomposition, drawing on the architecture of a production AI agent marketplace. The paper establishes that the marriage of market microstructure and category-theoretic planning yields provably correct orchestration with built-in economic incentives.

Keywords: AI planning, natural transformation, order book, profunctor, escrow monad, market microstructure, task decomposition, reputation systems, Elixir/OTP, agent marketplace

1 Introduction

Every operating system has a process that runs first and orchestrates everything else. In Unix, this is `init` (PID 1). In modern Linux, `systemd` manages service dependencies,

socket activation, and lifecycle transitions. In Kubernetes, the scheduler assigns pods to nodes based on resource requests, affinity rules, and priority classes. These systems solve the same fundamental problem: given a set of work items and a set of executors, determine *who does what, when, and how.*

An AI operating system faces this problem at a higher level of abstraction. The "processes" are not deterministic programs but stochastic agents with heterogeneous capabilities, varying reliability, and economic incentives. The "scheduler" must not only assign tasks but must negotiate prices, hold funds in escrow, verify quality, and update reputations. In short, the planner engine of an AI OS is simultaneously:

(i) A **task decomposer** that breaks complex jobs into dependency graphs of subtasks;

(ii) A **marketplace** where agents bid on work and clients select providers;

(iii) A **financial system** managing credits, escrow, and revenue distribution;

(iv) A **reputation engine** that scores quality and prevents gaming.

This paper provides a unified categorical framework for all four aspects. The central insight is that planning is a *natural transformation* between two functors: one that maps jobs to their requirements, and another that maps agents to their capabilities. The naturality condition ensures that planning decisions are consistent—changing the decomposition of a job and then matching is the same as matching and then decomposing the matched components.

The marketplace aspect is captured by an *order book* modeled as a profunctor. Just as in financial markets, buy orders (task requests) and sell orders (agent proposals) accumulate until a matching occurs. The escrow mechanism, which holds client credits until work is verified, satisfies the monad laws, providing algebraic guarantees of financial consistency. Market clearing—the moment when a proposal is accepted and a contract is formed—is characterized as a colimit in the transaction category.

We ground every abstraction in a concrete system: the AgentHero marketplace, where jobs follow the lifecycle `draft` → `open` → `in_progress` → `review`/`completed`/`cancelled`/`disputed`, operators submit proposals with execution plans and confidence scores, and atomic credit operations via Supabase RPC prevent race conditions. We provide a complete Elixir/OTP implementation demonstrating that the categorical structure maps directly to production-quality OTP patterns.

Contributions.

1. A formal market category $\mathcal{M}$ with participants as objects and transactions as morphisms, with the order book as a profunctor (**??**).
2. The planner as a natural transformation $\eta : F \Rightarrow G$ between task and capability functors, with a proof that naturality implies planning consistency (**??**).
3. Functorial job decomposition into DAGs with topological execution ordering (**??**).
4. The escrow monad with proofs of the monad laws guaranteeing financial consistency (**??**).
5. A 6-dimensional reputation system with convergence guarantees (**??**).
6. A complete Elixir/OTP implementation with GenServer order books, Mnesia escrow, and supervision trees (**??**).

Series context. This is Part IV of a five-part series on "The AI Operating System." Part I [?] introduced the agent OS kernel and type-theoretic foundations. Part II [?] formalized the memory layer as a sheaf over temporal topologies. Part III [?] modeled the tool interface via profunctors and Kan extensions. This paper addresses the planner engine. Part V will present the runtime and operational semantics.

2 Background

2.1 Process Orchestration in Classical Operating Systems

The evolution of process orchestration follows a clear trajectory of increasing abstraction:

Unix init. The original `init` process reads `/etc/inittab` and spawns services in a fixed order determined by runlevels. Dependencies are implicit in the ordering of startup scripts. This approach is inherently sequential and fragile—a slow service blocks all subsequent ones.

systemd. Poettering and Sievers' `systemd` [?] introduced a declarative dependency graph. Each unit file specifies `Requires`, `After`, `Wants`, and `Conflicts` relationships. The dependency graph is topologically sorted, and independent services start in parallel. Socket activation allows services to start on-demand. This is a significant advance: the scheduler reasons about a DAG of dependencies rather than a linear sequence.

Kubernetes. The Kubernetes scheduler [?] operates at the cluster level. It maintains a queue of unscheduled pods and iterates through a pipeline of *predicates* (hard constraints: resource availability, node affinity, taints/tolerations) and *priorities* (soft preferences: load balancing, data locality). The scheduler is extensible via scheduling frameworks and plugins. Critically, Kubernetes separates the *desired state* (declared in manifests) from the *actual state* (observed by controllers), with reconciliation loops driving convergence.

From orchestration to markets. Each of these systems assumes a benevolent, centralized scheduler that allocates resources optimally. In an AI agent ecosystem, this assumption breaks down. Agents are autonomous, self-interested entities. They may misrepresent capabilities, deliver substandard work, or collude. The planner must therefore incorporate market mechanisms—pricing, bidding, escrow, reputation—that align individual incentives with system-wide efficiency.

2.2 Market Microstructure

Market microstructure theory [?, ?] studies how trading mechanisms affect price formation, liquidity, and information aggregation. Key concepts include:

Order books. A limit order book (LOB) maintains buy orders (bids) and sell orders (asks), each specifying a price and quantity. Orders are matched when a bid price meets or exceeds an ask price. The *spread* between the best bid and best ask reflects liquidity and information asymmetry.

Matching engines. Matching follows priority rules: price-time priority (best price first, then earliest arrival) is standard. Pro-rata matching allocates proportionally at the same price level. In our setting, "price" is the credit cost and "quantity" is a unit of agent work.

Market clearing. A market clears when supply equals demand at some price. In continuous markets, clearing occurs order-by-order. In call auctions, orders accumulate and clear simultaneously at a single price. Our order book supports both modes.

Mechanism design. The revelation principle [**?**] states that any mechanism can be replicated by a direct revelation mechanism where truthful reporting is incentive-compatible. The Vickrey-Clarke-Groves (VCG) mechanism [**?**, **?**, **?**] achieves efficiency and truthfulness for combinatorial auctions. We draw on these results to design anti-gaming mechanisms for agent reputation.

2.3 Category Theory Preliminaries

We assume familiarity with categories, functors, and natural transformations at the level of Mac Lane [**?**]. We briefly recall the structures central to this paper.

Definition 2.1 (Profunctor)**.** A *profunctor* $P : \mathbf{C}^{\mathrm{op}} \times \mathbf{D} \to \mathbf{Set}$ is a functor from the product of the opposite of $\mathbf{C}$ with $\mathbf{D}$ to the category of sets. It generalizes the notion of a relation between categories. For objects $c \in \mathbf{C}$ and $d \in \mathbf{D}$, $P(c, d)$ is the set of "heteromorphisms" from c to d.

Definition 2.2 (Monad)**.** A *monad* on a category $\mathbf{C}$ is a triple (T, μ, η) where $T : \mathbf{C} \to \mathbf{C}$ is an endofunctor, $\eta : \mathrm{id}_{\mathbf{C}} \Rightarrow T$ is the unit, and $\mu : T^2 \Rightarrow T$ is the multiplication, satisfying:

$$\mu \circ T\mu = \mu \circ \mu T \quad \text{(associativity)} \tag{1}$$

$$\mu \circ T\eta = \mu \circ \eta T = \mathrm{id}_T \quad \text{(unit laws)} \tag{2}$$

Definition 2.3 (Colimit)**.** Given a diagram $D : \mathbf{J} \to \mathbf{C}$, a *colimit* of D is an object $\operatorname{colim} D$ in $\mathbf{C}$ together with a cocone $\{\iota_j : D(j) \to \operatorname{colim} D\}_{j \in \mathbf{J}}$ that is universal: for any other cocone $\{\phi_j : D(j) \to X\}_{j \in \mathbf{J}}$, there exists a unique morphism $u : \operatorname{colim} D \to X$ with $u \circ \iota_j = \phi_j$ for all j.

3 Market Model — Formal Framework

We now construct the categorical framework for the agent marketplace. The fundamental insight is that a marketplace is a category whose objects are economic participants and whose morphisms are transactions between them.

3.1 The Market Category

Definition 3.1 (Market Category)**.** The *market category* $\mathcal{M}$ is defined as follows:

- **Objects:** Economic participants, partitioned into *clients* $\mathcal{B}$ (buyers), *operators* $\mathcal{S}$ (sellers/agents), and the *platform* π. Thus $\mathrm{Ob}(\mathcal{M}) = \mathcal{B} \sqcup \mathcal{S} \sqcup \{\pi\}$.
- **Morphisms:** A morphism $f : a \to b$ is a *transaction* transferring credits from participant a to participant b. Each morphism carries:

  - An amount $\mathrm{amt}(f) \in \mathbb{R}_{\geq 0}$;
  - A timestamp $\mathrm{time}(f) \in \mathbb{R}_{\geq 0}$;
  - A transaction type $\mathrm{type}(f) \in \{\texttt{escrow_hold}, \texttt{escrow_release}, \texttt{payment}, \texttt{refund}, \texttt{fee}\}$.

- **Composition:** Sequential transactions compose: if $f : a \to b$ and $g : b \to c$, then $g \circ f : a \to c$ with $\mathrm{amt}(g \circ f) = \mathrm{amt}(g)$ (the final transfer amount).
- **Identity:** The identity $\mathrm{id}_a : a \to a$ is the zero-transfer transaction.

Remark 3.2. The market category is enriched over $(\mathbb{R}_{\geq 0}, +, 0)$: the hom-sets carry a monoidal structure given by addition of transaction amounts. This enrichment captures the financial flow structure.

3.2 The Order Book as a Profunctor

In a classical limit order book, buy orders and sell orders are two-sided. We generalize this to a profunctor relating agents and tasks.

Definition 3.3 (Agent and Task Categories)**.** Define the *agent category* $\mathcal{A}$ and the *task category* $\mathcal{T}$ as follows:

- $\mathcal{A}$: Objects are agents (operators). A morphism $\alpha \to \alpha'$ exists when agent α' can simulate or subsume the capabilities of α (capability ordering).
- $\mathcal{T}$: Objects are tasks (jobs or subtasks). A morphism $\tau \to \tau'$ exists when task τ is a prerequisite of τ' (dependency ordering).

Definition 3.4 (Order Book Profunctor)**.** The *order book* is a profunctor

$$P : \mathcal{A}^{\mathrm{op}} \times \mathcal{T} \to \mathbf{Set}$$

where $P(\alpha, \tau)$ is the set of *proposals* (sell orders) from agent α for task τ. Each proposal $p \in P(\alpha, \tau)$ consists of:

$$p = (\texttt{execution_plan}, \texttt{estimated_credits}, \texttt{estimated_duration}, \texttt{confidence_score}) \tag{3}$$

The profunctor action is:

- **Contravariant in $\mathcal{A}$:** If $\alpha' \to \alpha$ (i.e., α subsumes α'), then any proposal from α' can be lifted to a proposal from α: $P(\alpha', \tau) \to P(\alpha, \tau)$.
- **Covariant in $\mathcal{T}$:** If $\tau \to \tau'$ (i.e., τ is a prerequisite of τ'), then completing τ enables proposals for τ': there is an induced map $P(\alpha, \tau) \to P(\alpha, \tau')$.

Dually, we define the *demand side*:

Definition 3.5 (Demand Profunctor)**.** The *demand profunctor* is

$$Q : \mathcal{B}^{\mathrm{op}} \times \mathcal{T} \to \mathbf{Set}$$

where $Q(b, \tau)$ is the set of job postings by client b for task τ, each specifying a budget ceiling, required capabilities, and deadline.

3.3 Market Clearing as a Colimit

When a client accepts a proposal, we form a *contract*. This acceptance event is characterized as a colimit.

Definition 3.6 (Transaction Diagram)**.** Given a task τ with demand $q \in Q(b, \tau)$ and proposals $\{p_i \in P(\alpha_i, \tau)\}_{i \in I}$, the *transaction diagram* is the functor $D : \mathbf{J} \to \mathcal{M}$ where:

- $\mathbf{J}$ is the index category with objects $\{b, \alpha_1, \ldots, \alpha_n, \pi\}$ and morphisms representing potential transactions;
- D maps each object to the corresponding participant in $\mathcal{M}$ and each morphism to a proposed transaction.

Theorem 3.7 (Market Clearing as Colimit)**.** *Let $D : \mathbf{J} \to \mathcal{M}$ be a transaction diagram for task τ. The market clearing event—where one proposal p_k is accepted and all others $\{p_i\}_{i \neq k}$ are rejected—is the colimit* $\operatorname{colim} D$ *in $\mathcal{M}$. The universal cocone consists of:*

1. *The escrow morphism $e : b \to \pi$ with* $\mathrm{amt}(e) =$ `estimated_credits`(p_k)*;*
2. *The contract formation $c : \pi \to \alpha_k$ (pending work completion);*
3. *Rejection morphisms $r_i : \alpha_i \to \alpha_i$ (identity, no transaction) for $i \neq k$.*

Proof. We verify the universal property. Let $\{e, c, r_i\}$ be the proposed cocone. For any other cocone $\{\phi_j\}_{j \in \mathbf{J}}$ over the same diagram, the unique mediating morphism $u : \operatorname{colim} D \to X$ is determined by the contract terms: the escrow amount uniquely determines the payment flow, and the accepted proposal uniquely determines the executor. The rejection morphisms are identities, so they factor trivially. The uniqueness of u follows from the budget ceiling constraint—only one allocation of the client's credits is consistent with accepting exactly one proposal.

To show existence, observe that accepting proposal p_k induces:

$$\texttt{holdEscrow}(b, \texttt{estimated_credits}(p_k), \texttt{contract_id}) \tag{4}$$

which atomically decrements the client's balance and creates an escrow record. This atomic operation, implemented via Supabase RPC, ensures that the colimit exists and is unique (no double-spending). □

Corollary 3.8. *In the colimit construction, accepting proposal p_k necessarily rejects all other proposals $\{p_i\}_{i \neq k}$. This is the universal property at work: the cocone maps are uniquely determined.*

This matches the AgentHero implementation exactly: when a client accepts one proposal, all others are auto-rejected, and a contract is created with escrowed credits.

3.4 The Escrow Monad

Definition 3.9 (Credit Category)**.** The *credit category* $\mathcal{C}$ has:

- Objects: credit balances (b, n) where b is a participant and $n \in \mathbb{R}_{\geq 0}$ is the balance;
- Morphisms: balance transitions $(b, n) \to (b, m)$ representing credit/debit operations.

Definition 3.10 (Escrow Monad)**.** The *escrow monad* $E : \mathcal{C} \to \mathcal{C}$ is the endofunctor defined by:

$$E(b, n) = (b, n - h, h) \quad \text{where } h \text{ is the held amount} \tag{5}$$

$$\eta_{(b,n)} : (b, n) \to E(b, n) = (b, n, 0) \quad \text{(unit: no hold)} \tag{6}$$

$$\mu_{(b,n,h)} : E(E(b, n)) \to E(b, n) \quad \text{(join: flatten nested holds)} \tag{7}$$

The monadic operations correspond to:

- **Unit (η):** Initialize escrow with zero hold. "return" in Haskell.
- **Bind ($\gg=$):** Sequence escrow operations. Holding credits for contract c_1 and then for c_2 composes into a single escrow state.
- **Join (μ):** Flatten $E(E(b, n))$ to $E(b, n)$—settle an escrow-within-escrow (e.g., subcontracting).

4 Planning as Natural Transformation

We now formalize the planner as the central natural transformation connecting what needs to be done with who can do it.

4.1 The Task Functor

Definition 4.1 (Requirement Category)**.** The *requirement category* $\mathcal{R}$ has:

- Objects: requirement specifications $r = (\texttt{capabilities}, \texttt{constraints}, \texttt{budget}, \texttt{deadline})$;
- Morphisms: requirement refinements $r \to r'$ where r' is a stricter specification (more capabilities required, tighter constraints, etc.).

Definition 4.2 (Task Functor)**.** The *task functor* $F : \mathcal{T} \to \mathcal{R}$ maps each task to its requirement specification:

$$F(\tau) = (\texttt{required_capabilities}(\tau), \texttt{constraints}(\tau), \texttt{budget}(\tau), \texttt{deadline}(\tau)) \tag{8}$$

For a morphism $\tau \to \tau'$ (dependency), F maps it to the induced requirement refinement: if τ is a prerequisite of τ', then the requirements of τ' refine those of τ (building on completed prerequisites).

4.2 The Capability Functor

Definition 4.3 (Capability Category)**.** The *capability category* $\Cap$ has:

- Objects: capability profiles $k = (\texttt{skills}, \texttt{capacity}, \texttt{pricing}, \texttt{reputation})$;
- Morphisms: capability enhancements $k \to k'$ where k' has additional skills or capacity.

Definition 4.4 (Capability Functor)**.** The *capability functor* $G : \mathcal{A} \to \Cap$ maps each agent to its capability profile:

$$G(\alpha) = (\texttt{skills}(\alpha), \texttt{capacity}(\alpha), \texttt{pricing_model}(\alpha), \texttt{reputation}(\alpha)) \tag{9}$$

where $\texttt{pricing_model} \in \{\texttt{per_task}, \texttt{hourly}, \texttt{per_token}\}$.

4.3 The Planner Natural Transformation

Definition 4.5 (Planner)**.** The *planner* is a natural transformation

$$\eta : F \Rightarrow G \circ M$$

where $M : \mathcal{T} \to \mathcal{A}$ is the matching functor (assigning agents to tasks). For each task τ, the component $\eta_\tau : F(\tau) \to G(M(\tau))$ witnesses that the assigned agent's capabilities satisfy the task's requirements.

In the expanded form, this is a natural transformation in the following commutative diagram. For every morphism $f : \tau \to \tau'$ in $\mathcal{T}$ (a dependency relation):

$$\begin{tikzcd}
F(\tau) \arrow[r, "\eta_\tau"] \arrow[d, "F(f)"'] & G(M(\tau)) \arrow[d, "G(M(f))"] \\
F(\tau') \arrow[r, "\eta_{\tau'}"'] & G(M(\tau'))
\end{tikzcd}$$

Theorem 4.6 (Planning Consistency)**.** *Let $\eta : F \Rightarrow G \circ M$ be the planner natural transformation. The naturality condition*

$$G(M(f)) \circ \eta_\tau = \eta_{\tau'} \circ F(f)$$

for every dependency $f : \tau \to \tau'$ guarantees planning consistency*: the following two procedures yield the same result:*

1. *Refine the requirements of τ to those of τ' (via $F(f)$), then match τ' to an agent (via $\eta_{\tau'}$).*
2. *Match τ to an agent (via η_τ), then enhance the agent's capability commitment from that of τ to that of τ' (via $G(M(f))$).*

Proof. This is precisely the naturality square. The condition states that the diagram commutes, i.e., the two paths from $F(\tau)$ to $G(M(\tau'))$ are equal. Concretely, this means that planning decisions are stable under requirement refinement. If task τ is a prerequisite of τ', then the agent assigned to handle the chain $\tau \to \tau'$ must be capable of handling the refined requirements. This prevents the situation where an agent is assigned to τ but cannot handle the downstream task τ' that depends on τ's output. □

Corollary 4.7 (Optimal Matching)**.** *The optimal matching functor M^* minimizes the cost functional*

$$M^* = \arg\min_M \sum_{\tau \in \mathrm{Ob}(\mathcal{T})} \texttt{cost}(\eta_\tau)$$

subject to the naturality constraint $G(M(f)) \circ \eta_\tau = \eta_{\tau'} \circ F(f)$ for all dependencies f, where $\texttt{cost}(\eta_\tau)$ incorporates the estimated credits, inverse confidence score, and reputation discount:

$$\texttt{cost}(\eta_\tau) = \frac{\texttt{estimated_credits}(M(\tau), \tau)}{1 + \texttt{confidence_score}(M(\tau), \tau) \cdot \texttt{reputation}(M(\tau))}$$

Proof. The cost functional is well-defined on the space of natural transformations $\eta : F \Rightarrow G \circ M$ because the naturality constraint restricts M to those matchings that respect task dependencies. The minimum exists because the set of agents is finite, the tasks are finite, and the cost functional is bounded below by zero. □

4.4 The Matching Process

In the AgentHero system, matching proceeds through the following stages, each of which we interpret categorically:

1. **Job posting (draft → open):** A client creates a job with requirements $F(\tau)$. This defines the left-hand side of the naturality square.
2. **Proposal submission:** Operators inspect $F(\tau)$ and submit proposals $p \in P(\alpha, \tau)$. Each proposal is a candidate component $\eta_\tau^{(\alpha)} : F(\tau) \to G(\alpha)$.
3. **Proposal evaluation:** The client evaluates proposals. This is a selection over the fiber $P(-, \tau) = \coprod_\alpha P(\alpha, \tau)$.
4. **Acceptance (open → in_progress):** The client accepts proposal p_k from agent α_k. This selects the component $\eta_\tau = \eta_\tau^{(\alpha_k)}$ and defines $M(\tau) = \alpha_k$. All other proposals are auto-rejected (**??**).
5. **Contract creation:** The acceptance triggers `holdEscrow`(b, `rate_credits`, `contract_id`), applying the escrow monad unit.
6. **Execution and review (in_progress → review):** The agent executes the task. Upon completion, the work enters review.
7. **Settlement (review → completed):** Approval releases escrow via the monad join, distributing credits: 70% operator, 15% platform, 15% LLM reserve.

5 Job Decomposition

Complex jobs must be broken into manageable subtasks. The AgentHero orchestrator performs this decomposition, producing a directed acyclic graph (DAG) of subtasks with dependency edges.

5.1 The Decomposition Functor

Definition 5.1 (DAG Category)**.** Let **DAG** be the category whose objects are directed acyclic graphs (V, E) with V a finite set of vertices (subtasks) and $E \subseteq V \times V$ the edge set (dependencies). A morphism $\phi : (V, E) \to (V', E')$ is a graph homomorphism preserving the partial order.

Definition 5.2 (Decomposition Functor)**.** The *decomposition functor* $\Delta : \mathcal{T} \to \mathbf{DAG}$ maps each task τ to its decomposition graph $\Delta(\tau) = (V_\tau, E_\tau)$, where:

- $V_\tau = \{\sigma_1, \ldots, \sigma_n\}$ are the subtasks;
- $(\sigma_i, \sigma_j) \in E_\tau$ iff σ_i must complete before σ_j begins;
- For a dependency $f : \tau \to \tau'$, the morphism $\Delta(f) : \Delta(\tau) \to \Delta(\tau')$ maps terminal subtasks of $\Delta(\tau)$ to initial subtasks of $\Delta(\tau')$.

Proposition 5.3 (Decomposition Preserves Job Semantics)**.** *The decomposition functor Δ is faithful: distinct tasks have distinct decompositions, and the dependency structure is preserved. Formally:*

1. *Δ preserves identities:* $\Delta(\mathrm{id}_\tau) = \mathrm{id}_{\Delta(\tau)}$.
2. *Δ preserves composition:* $\Delta(g \circ f) = \Delta(g) \circ \Delta(f)$ *for composable dependencies* f, g.

Consequently, executing the decomposed subtasks in any topological order that respects E_τ is equivalent to executing the original task τ.

Proof. Preservation of identities: the identity dependency $\mathrm{id}_\tau : \tau \to \tau$ maps to the identity graph homomorphism on $\Delta(\tau)$.

Preservation of composition: let $f : \tau_1 \to \tau_2$ and $g : \tau_2 \to \tau_3$. Then $\Delta(f)$ connects terminal subtasks of $\Delta(\tau_1)$ to initial subtasks of $\Delta(\tau_2)$, and $\Delta(g)$ connects terminal subtasks of $\Delta(\tau_2)$ to initial subtasks of $\Delta(\tau_3)$. The composition $\Delta(g) \circ \Delta(f)$ is the graph homomorphism that connects terminal subtasks of $\Delta(\tau_1)$ through $\Delta(\tau_2)$ to initial subtasks of $\Delta(\tau_3)$, which is exactly $\Delta(g \circ f)$.

Faithfulness: distinct dependencies induce distinct graph homomorphisms because the subtask structure carries strictly more information than the coarse dependency. □

5.2 Topological Sort for Execution Ordering

Given a DAG $\Delta(\tau) = (V_\tau, E_\tau)$, we compute a topological ordering $\sigma_{i_1}, \sigma_{i_2}, \ldots, \sigma_{i_n}$ such that $(\sigma_{i_j}, \sigma_{i_k}) \in E_\tau \implies j < k$.

Algorithm 1: Topological Sort with Parallel Barrier Detection

Input: DAG (V, E) with $|V| = n$
Output: Execution schedule $S = [L_1, L_2, \ldots, L_m]$ where each L_i is a parallel execution level

```
Compute in-degrees: ∀v ∈ V, deg⁻(v) = |{u : (u, v) ∈ E}|;
L_0 ← {v ∈ V : deg⁻(v) = 0} // sources: no dependencies
S ← [L_0];
i ← 0;
while ⋃_{j≤i} L_j ≠ V do
    L_{i+1} ← ∅;
    foreach v ∈ L_i do
        foreach (v, w) ∈ E do
            deg⁻(w) ← deg⁻(w) − 1;
            if deg⁻(w) = 0 then
                L_{i+1} ← L_{i+1} ∪ {w};
            end
        end
    end
    S ← S ∪ [L_{i+1}];
    i ← i + 1;
end
return S
```

Each level L_i consists of subtasks that can execute in parallel—their dependencies are all satisfied by the completion of levels $L_0, \ldots, L_{i-1}$. The transition from L_i to L_{i+1} is a *barrier synchronization*: all tasks in L_i must complete before any task in L_{i+1} begins.

Proposition 5.4 (Optimality of Level-Based Parallelism)**.** *The number of levels m in* **??** *equals the length of the longest path in the DAG. This is the theoretical minimum number of sequential steps required, and is therefore optimal for parallel execution.*

Proof. Each level contains all vertices whose longest incoming path has the same length. The critical path (longest path from any source to any sink) determines m. No schedule can execute in fewer than m steps, because the critical path tasks are sequentially dependent. □

5.3 Integration with Inngest Durable Execution

In the AgentHero implementation, each subtask execution is wrapped in an Inngest function, providing:

- **Event-driven triggers:** Subtask σ_j fires an event upon completion; dependent subtasks σ_k are triggered by this event.
- **Automatic retries:** Transient failures (LLM timeouts, rate limits) are retried with exponential backoff.
- **Idempotency:** Each subtask execution is idempotent, preventing duplicate work on retry.
- **Observability:** The execution trace is a reified version of the DAG with timing information.

The categorical interpretation is a functor from the DAG category to the category of Inngest functions:

$$\Phi : \mathbf{DAG} \to \mathbf{Inngest}$$

where **Inngest** is the category of event-driven durable functions with composition given by event chaining.

6 Financial Mechanics

6.1 The Escrow Monad in Detail

We now prove that the escrow mechanism satisfies the monad laws, providing algebraic guarantees of financial consistency.

Definition 6.1 (Escrow Endofunctor)**.** Define $E : \mathcal{C} \to \mathcal{C}$ on objects by

$$E(b, n) = (b, n_{\text{available}}, n_{\text{held}})$$

where $n = n_{\text{available}} + n_{\text{held}}$, and on morphisms by

$$E(f : (b, n) \to (b, m)) = (f_{\text{avail}} : n_{\text{available}} \to m_{\text{available}},\ f_{\text{held}} : n_{\text{held}} \to m_{\text{held}})$$

preserving the invariant $n_{\text{available}} + n_{\text{held}} = m_{\text{available}} + m_{\text{held}}$ (credits are conserved).

Definition 6.2 (Escrow Unit and Multiplication)**.** The *unit* $\eta : \mathrm{id}_{\mathcal{C}} \Rightarrow E$ is:

$$\eta_{(b,n)} : (b, n) \mapsto (b, n, 0) \quad \text{(no credits held)}$$

The *multiplication* $\mu : E^2 \Rightarrow E$ flattens nested escrow:

$$\mu_{(b,n)} : E(E(b, n)) = (b, n_{\text{avail}}, (n_{\text{held_outer}}, n_{\text{held_inner}})) \mapsto (b, n_{\text{avail}}, n_{\text{held_outer}} + n_{\text{held_inner}})$$

This corresponds to settling a subcontract escrow into the parent escrow.

Theorem 6.3 (Escrow Monad Laws)**.** *The triple* (E, η, μ) *satisfies the monad laws:*

1. ***Left unit:*** $\mu \circ \eta E = \mathrm{id}_E$
2. ***Right unit:*** $\mu \circ E\eta = \mathrm{id}_E$
3. ***Associativity:*** $\mu \circ \mu E = \mu \circ E\mu$

Proof. **Left unit.** Starting from $E(b, n) = (b, n_a, n_h)$:

$$\begin{aligned}(\mu \circ \eta E)(b, n_a, n_h) &= \mu(\eta(b, n_a, n_h)) && (10)\\ &= \mu(b, (n_a, n_h), 0) && (11)\\ &= (b, n_a, n_h + 0) = (b, n_a, n_h) = \mathrm{id}_E(b, n_a, n_h) && (12)\end{aligned}$$

Right unit. Starting from $E(b, n) = (b, n_a, n_h)$:

$$\begin{aligned}(\mu \circ E\eta)(b, n_a, n_h) &= \mu(E(\eta)(b, n_a, n_h)) && (13)\\ &= \mu(b, n_a, (n_h, 0)) && (14)\\ &= (b, n_a, n_h + 0) = (b, n_a, n_h) = \mathrm{id}_E(b, n_a, n_h) && (15)\end{aligned}$$

Associativity. Starting from $E^3(b, n)$ with three levels of holding (h_1, h_2, h_3):

$$(\mu \circ \mu E)(b, n_a, (h_1, (h_2, h_3))) = \mu(b, n_a, (h_1, h_2 + h_3)) = (b, n_a, h_1 + h_2 + h_3) \quad (16)$$
$$(\mu \circ E\mu)(b, n_a, (h_1, (h_2, h_3))) = \mu(b, n_a, (h_1 + h_2, h_3)) = (b, n_a, h_1 + h_2 + h_3) \quad (17)$$

Both paths yield the same result, since addition is associative. □

6.2 Revenue Split as Natural Transformation

Upon contract completion, escrowed credits are distributed:

Definition 6.4 (Revenue Split)**.** The *revenue split* is a natural transformation $\rho : E \Rightarrow D$ where $D : \mathcal{C} \to \mathcal{C}^3$ is the distribution functor mapping a single credit pool to a triple:

$$D(n_{\text{held}}) = (0.70 \cdot n_{\text{held}},\ 0.15 \cdot n_{\text{held}},\ 0.15 \cdot n_{\text{held}})$$

corresponding to (operator payment, platform fee, LLM reserve).

The three pricing models affect how n_{held} is computed:

$$\begin{aligned}n_{\text{held}}^{\texttt{per_task}} &= \texttt{fixed_price} && (18)\\ n_{\text{held}}^{\texttt{hourly}} &= \texttt{rate_credits} \times \lceil \texttt{hours_worked} \rceil && (19)\\ n_{\text{held}}^{\texttt{per_token}} &= \texttt{rate_per_token} \times \texttt{tokens_consumed} && (20)\end{aligned}$$

In all cases, $n_{\text{held}} \leq \texttt{budget_ceiling}$ to prevent overspend.

6.3 Atomic Operations and Race Condition Prevention

The escrow operations must be atomic to prevent race conditions. Consider two concurrent contracts attempting to hold credits from the same client balance:

Definition 6.5 (Atomic Credit Operations)**.** An *atomic credit operation* is a morphism in $\mathcal{C}$ that is implemented via a database-level serializable transaction. In the AgentHero system, this is a Supabase RPC call:

```
adminClient.rpc('hold_escrow', {
  p_client_id: client_id,
  p_amount: rate_credits,
  p_contract_id: contract_id
})
```

The RPC function executes within a PostgreSQL transaction with `SERIALIZABLE` isolation, ensuring:

1. The balance check and deduction are atomic (no TOCTOU race);
2. Negative balances are prevented by a `CHECK(balance >= 0)` constraint;
3. Concurrent operations are serialized by the database.

Theorem 6.6 (Financial Consistency)**.** *Under the escrow monad with atomic credit operations, the following invariants hold:*

1. ***Conservation:*** *The total credits in the system (available + held + distributed) is constant.*
2. ***Non-negativity:*** *No participant's available balance is ever negative.*
3. ***Escrow completeness:*** *Every held amount is eventually either released (completed) or refunded (cancelled/disputed).*

Proof. **Conservation** follows from the monad laws: η preserves totals (holds move credits from available to held, not creating or destroying them), and μ flattens without changing totals.

Non-negativity is enforced by the database constraint `CHECK(balance >= 0)`. The atomic RPC rolls back if this constraint would be violated.

Escrow completeness follows from the job lifecycle being a finite state machine with terminal states $\{\texttt{completed}, \texttt{cancelled}, \texttt{disputed}\}$. Each terminal state triggers either release or refund. With Inngest durable execution, liveness is guaranteed: the event-driven pipeline will eventually reach a terminal state (possibly via timeout). □

7 Reputation and Trust

7.1 Six-Dimensional Quality Scoring

After each job completion, the evaluator computes a quality vector capturing multiple dimensions of performance.

Definition 7.1 (Quality Vector)**.** The *quality vector* for a completed job is $\mathbf{q} \in [0,1]^6$ with components:

$$\mathbf{q} = (q_{\text{accuracy}},\ q_{\text{completeness}},\ q_{\text{timeliness}},\ q_{\text{communication}},\ q_{\text{efficiency}},\ q_{\text{innovation}}) \tag{21}$$

where:

- q_{accuracy}: correctness of deliverables against requirements;
- $q_{\text{completeness}}$: fraction of requirements addressed;
- $q_{\text{timeliness}}$: $\max(0, 1 - \frac{\text{actual_duration} - \text{estimated_duration}}{\text{deadline}})$;
- $q_{\text{communication}}$: responsiveness and clarity (client-rated);
- $q_{\text{efficiency}}$: $1 - \frac{\text{actual_credits} - \text{estimated_credits}}{\text{budget_ceiling}}$;
- $q_{\text{innovation}}$: exceeding requirements or novel approaches (evaluator-scored).

Definition 7.2 (Reputation Score)**.** The *reputation score* of agent α after N completed jobs is the exponentially-weighted inner product:

$$R(\alpha) = \frac{\sum_{i=1}^{N} \lambda^{N-i} \langle \mathbf{w}, \mathbf{q}_i \rangle}{\sum_{i=1}^{N} \lambda^{N-i}} \tag{22}$$

where $\mathbf{w} \in \mathbb{R}^6_{\geq 0}$ with $\|\mathbf{w}\|_1 = 1$ is a weight vector (default: uniform $\mathbf{w} = (\frac{1}{6}, \ldots, \frac{1}{6})$), and $\lambda \in (0,1)$ is the decay factor (default: $\lambda = 0.95$), giving more weight to recent performance.

7.2 Reputation as a Functor

Definition 7.3 (Reputation Functor)**.** The *reputation functor* $\mathcal{R} : \mathcal{A} \to [0,1]$ maps each agent to its reputation score and each capability enhancement morphism $\alpha \to \alpha'$ to the induced reputation update. The codomain $[0,1]$ is viewed as a poset category (with the usual ordering $\leq$).

Proposition 7.4 (Reputation Convergence)**.** *Under honest participation (i.e., quality vectors $\mathbf{q}_i$ are drawn i.i.d. from a distribution with mean $\boldsymbol{\mu}$ and finite variance), the reputation score converges:*

$$R(\alpha) \xrightarrow{N \to \infty} \langle \mathbf{w}, \boldsymbol{\mu} \rangle$$

almost surely.

Proof. The exponentially-weighted average is a consistent estimator of $\langle \mathbf{w}, \boldsymbol{\mu} \rangle$. Writing $X_i = \langle \mathbf{w}, \mathbf{q}_i \rangle$, we have $\mathbb{E}[X_i] = \langle \mathbf{w}, \boldsymbol{\mu} \rangle$ and

$$R(\alpha) = \frac{\sum_{i=1}^{N} \lambda^{N-i} X_i}{\sum_{i=1}^{N} \lambda^{N-i}} = \frac{1-\lambda}{1-\lambda^N} \sum_{i=1}^{N} \lambda^{N-i} X_i$$

As $N \to \infty$, $\frac{1-\lambda}{1-\lambda^N} \to (1-\lambda)$ and $\sum_{i=1}^{N} \lambda^{N-i} X_i$ is a geometrically-weighted sum. By the strong law of large numbers for weighted averages (see [**?**]), convergence holds a.s. provided $\text{Var}(X_i) < \infty$, which is guaranteed since $X_i \in [0,1]$. □

7.3 Anti-Gaming Mechanisms

Gaming the reputation system—inflating scores through fake jobs, collusion, or strategic behavior—is modeled as a constraint satisfaction problem.

Definition 7.5 (Gaming Detection)**.** An agent α is flagged for *suspected gaming* if any of the following anomaly detectors fire:

1. **Self-dealing:** α and the client share a wallet address or IP cluster.
2. **Score inflation:** The quality variance $\mathrm{Var}(\mathbf{q}_1, \ldots, \mathbf{q}_N) < \epsilon$ for suspiciously many jobs (all scores are nearly identical).
3. **Rapid cycling:** Jobs complete in less than $\delta_{\min}$ time with perfect scores.
4. **Collusion ring:** A clique of agents and clients who only transact with each other, detected via graph analysis of the transaction subgraph.
5. **Reputation laundering:** Creating a new agent identity after accumulating negative reputation.

The trust score incorporates both reputation and anti-gaming signals:

Definition 7.6 (Trust Score)**.** The *trust score* is:

$$T(\alpha) = R(\alpha) \cdot (1 - \gamma(\alpha)) \cdot \min\left(1, \frac{N(\alpha)}{N_{\min}}\right)$$

where $\gamma(\alpha) \in [0, 1]$ is the gaming suspicion level, and the third factor is a "seasoning" term requiring a minimum number of jobs $N_{\min}$ before full trust is granted.

8 Elixir/OTP Implementation

We now present a production-quality Elixir implementation of the planner engine, demonstrating that the categorical abstractions map directly to OTP patterns. The implementation consists of five modules: the order book (GenServer with matching engine), the escrow system (GenServer with Mnesia transactions), the task decomposer (with supervision trees), the reputation engine, and the market module.

8.1 Application Architecture

The application follows standard OTP conventions with a supervision tree:

```
defmodule PlannerEngine.Application do
  use Application

  def start(_type, _args) do
    children = [
      {PlannerEngine.OrderBook, []},
      {PlannerEngine.Escrow, []},
      {PlannerEngine.Reputation, []},
      {PlannerEngine.Market, []}
```

```
    ]

    opts = [strategy: :one_for_one,
            name: PlannerEngine.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

Listing 1: Application supervision tree

The `:one_for_one` strategy ensures that if a single component crashes, only that component is restarted—the others continue operating. This is the let-it-crash philosophy of Erlang/OTP applied to financial infrastructure.

8.2 Order Book GenServer

The order book maintains buy and sell sides, with price-time priority matching:

```
defmodule PlannerEngine.OrderBook do
  use GenServer

  @type proposal :: %{
    agent_id: String.t(),
    task_id: String.t(),
    execution_plan: String.t(),
    estimated_credits: non_neg_integer(),
    estimated_duration: non_neg_integer(),
    confidence_score: float(),
    timestamp: DateTime.t()
  }

  defstruct proposals: %{}, demands: %{}, matches: []

  # Submit proposal (sell order)
  def submit_proposal(proposal) do
    GenServer.call(__MODULE__, {:submit_proposal, proposal})
  end

  # Post demand (buy order)
  def post_demand(demand) do
    GenServer.call(__MODULE__, {:post_demand, demand})
  end

  def handle_call({:submit_proposal, proposal}, _from, state) do
    task_id = proposal.task_id
    proposals = Map.update(
      state.proposals, task_id, [proposal],
      &insert_by_priority(&1, proposal)
    )
    new_state = %{state | proposals: proposals}
    {:reply, :ok, maybe_match(new_state, task_id)}
```

```
  end

  # Price-time priority insertion
  defp insert_by_priority(existing, new) do
    Enum.sort_by([new | existing], fn p ->
      {p.estimated_credits, p.timestamp}
    end)
  end

  # Match when demand meets supply
  defp maybe_match(state, task_id) do
    with demands when demands != [] <-
           Map.get(state.demands, task_id, []),
         [best | _rest] <-
           Map.get(state.proposals, task_id, []),
         true <- best.estimated_credits <=
                   hd(demands).budget_ceiling do
      execute_match(state, task_id, hd(demands), best)
    else
      _ -> state
    end
  end
end
```

Listing 2: Order book GenServer (simplified)

The matching engine implements the profunctor action: proposals are indexed by agent and task, and matching selects the optimal element from the fiber $P(-, \tau)$.

8.3 Escrow GenServer with Mnesia

The escrow module uses Mnesia for transactional guarantees:

```
defmodule PlannerEngine.Escrow do
  use GenServer

  # Monad unit: initialize escrow (hold credits)
  @spec hold(String.t(), non_neg_integer(), String.t())
    :: {:ok, String.t()} | {:error, atom()}
  def hold(client_id, amount, contract_id) do
    GenServer.call(__MODULE__,
      {:hold, client_id, amount, contract_id})
  end

  # Monad bind: sequence escrow operations
  @spec bind(String.t(), (map() -> {:ok, map()}))
    :: {:ok, map()} | {:error, atom()}
  def bind(escrow_id, f) do
    GenServer.call(__MODULE__, {:bind, escrow_id, f})
  end

```

```
  # Monad join: settle (release or refund)
  @spec settle(String.t(), :release | :refund)
    :: {:ok, map()} | {:error, atom()}
  def settle(escrow_id, action) do
    GenServer.call(__MODULE__,
      {:settle, escrow_id, action})
  end

  def handle_call({:hold, client_id, amount, cid},
                  _from, state) do
    result = :mnesia.transaction(fn ->
      case :mnesia.read(:balances, client_id) do
        [{:balances, ^client_id, balance}]
          when balance >= amount ->
          :mnesia.write({:balances, client_id,
                         balance - amount})
          escrow_id = generate_id()
          :mnesia.write({:escrows, escrow_id,
            client_id, amount, cid, :held})
          escrow_id
        _ ->
          :mnesia.abort(:insufficient_funds)
      end
    end)

    case result do
      {:atomic, eid} -> {:reply, {:ok, eid}, state}
      {:aborted, reason} ->
        {:reply, {:error, reason}, state}
    end
  end
end
```

Listing 3: Escrow monad implementation

The Mnesia transaction ensures atomicity: the balance check and deduction happen within a single transaction, preventing the TOCTOU race condition described in **??**.

8.4 Task Decomposition with Supervision Trees

```
defmodule PlannerEngine.Decomposer do
  @type subtask :: %{
    id: String.t(),
    parent_task_id: String.t(),
    description: String.t(),
    dependencies: [String.t()],
    status: :pending | :running | :completed | :failed
  }

  @type dag :: %{
    task_id: String.t(),
```

```
    subtasks: %{String.t() => subtask()},
    edges: [{String.t(), String.t()}]
  }

  @spec decompose(map()) :: {:ok, dag()} | {:error, term()}
  def decompose(task) do
    subtasks = analyze_and_split(task)
    edges = infer_dependencies(subtasks)

    dag = %{
      task_id: task.id,
      subtasks: Map.new(subtasks, &{&1.id, &1}),
      edges: edges
    }

    case validate_dag(dag) do
      :ok -> {:ok, dag}
      {:error, :cycle} -> {:error, :cyclic_dependency}
    end
  end

  @spec topological_sort(dag()) :: [[String.t()]]
  def topological_sort(dag) do
    in_degrees = compute_in_degrees(dag)

    sources =
      for {id, 0} <- in_degrees, do: id

    build_levels(dag, sources, in_degrees, [])
  end

  defp build_levels(_dag, [], _degrees, levels),
    do: Enum.reverse(levels)
  defp build_levels(dag, current_level, degrees, levels) do
    new_degrees =
      Enum.reduce(current_level, degrees, fn id, acc ->
        dag.edges
        |> Enum.filter(fn {from, _to} -> from == id end)
        |> Enum.reduce(acc, fn {_from, to}, inner_acc ->
          Map.update!(inner_acc, to, &(&1 - 1))
        end)
      end)

    next_level =
      for {id, 0} <- new_degrees,
          id not in List.flatten([current_level | levels]),
          do: id

    build_levels(dag, next_level, new_degrees,
                 [current_level | levels])
  end
```

```
end
```

Listing 4: DAG-based task decomposition

The `topological_sort/1` function implements ??, returning a list of levels where each level can execute in parallel. The supervision tree spawns a `Task.Supervisor` for each level, running all subtasks concurrently and awaiting completion before proceeding to the next level.

8.5 Reputation Engine

```
defmodule PlannerEngine.Reputation do
  use GenServer

  @dimensions [:accuracy, :completeness, :timeliness,
               :communication, :efficiency, :innovation]
  @default_weights List.duplicate(1/6, 6)
  @decay_factor 0.95

  @spec compute_score(String.t()) :: float()
  def compute_score(agent_id) do
    GenServer.call(__MODULE__,
      {:compute_score, agent_id})
  end

  @spec record_quality(String.t(), [float()]) :: :ok
  def record_quality(agent_id, quality_vector)
    when length(quality_vector) == 6 do
    GenServer.cast(__MODULE__,
      {:record, agent_id, quality_vector})
  end

  @spec trust_score(String.t()) :: float()
  def trust_score(agent_id) do
    GenServer.call(__MODULE__,
      {:trust_score, agent_id})
  end

  def handle_call({:compute_score, agent_id},
                  _from, state) do
    history = Map.get(state.histories, agent_id, [])
    score = exponential_weighted_score(
              history, @default_weights, @decay_factor)
    {:reply, score, state}
  end

  defp exponential_weighted_score([], _w, _l), do: 0.5
  defp exponential_weighted_score(history, weights, lambda) do
    n = length(history)

```

```
    {weighted_sum, weight_total} =
      history
      |> Enum.with_index()
      |> Enum.reduce({0.0, 0.0}, fn {qv, i}, {ws, wt} ->
        decay = :math.pow(lambda, n - 1 - i)
        inner = Enum.zip(weights, qv)
                |> Enum.map(fn {w, q} -> w * q end)
                |> Enum.sum()
        {ws + decay * inner, wt + decay}
      end)

    weighted_sum / weight_total
  end
end
```

Listing 5: Six-dimensional reputation scoring

8.6 Market Module

```
defmodule PlannerEngine.Market do
  use GenServer

  @operator_share 0.70
  @platform_share 0.15
  @llm_reserve_share 0.15

  @type contract :: %{
    id: String.t(),
    client_id: String.t(),
    operator_id: String.t(),
    task_id: String.t(),
    escrow_id: String.t(),
    status: :active | :completed | :disputed,
    pricing_model: :per_task | :hourly | :per_token,
    rate_credits: non_neg_integer()
  }

  @spec clear_market(String.t()) :: {:ok, contract()}
    | {:error, term()}
  def clear_market(task_id) do
    GenServer.call(__MODULE__,
      {:clear_market, task_id})
  end

  @spec distribute_revenue(String.t()) :: {:ok, map()}
  def distribute_revenue(contract_id) do
    GenServer.call(__MODULE__,
      {:distribute, contract_id})
  end

```

```
  defp do_distribute(contract) do
    total = compute_total(contract)
    %{
      operator: round(total * @operator_share),
      platform: round(total * @platform_share),
      llm_reserve: round(total * @llm_reserve_share)
    }
  end

  defp compute_total(%{pricing_model: :per_task} = c),
    do: c.rate_credits
  defp compute_total(%{pricing_model: :hourly} = c),
    do: c.rate_credits * ceil(c.hours_worked)
  defp compute_total(%{pricing_model: :per_token} = c),
    do: c.rate_per_token * c.tokens_consumed
end
```

Listing 6: Market clearing and revenue distribution

9 Case Studies

9.1 AI Agent Marketplace: The AgentHero Model

The AgentHero platform implements the full framework described in this paper. We trace a concrete job through the categorical machinery.

Scenario. A client needs a web application tested. The job requires:

- Functional testing (Playwright-based E2E tests)
- Performance testing (k6 load tests)
- Security scanning (OWASP ZAP)
- Accessibility audit (axe-core)

Step 1: Task functor. The client posts the job (status: `draft`). The task functor F maps it to requirements:

$$F(\tau) = (\{\text{playwright}, \text{k6}, \text{zap}, \text{axe}\}, \{t \leq 4\text{h}\}, 5000 \text{ credits}, 24\text{h deadline})$$

Step 2: Job decomposition. The orchestrator applies the decomposition functor Δ:

$$\Delta(\tau) = \begin{cases} V = \{\sigma_{\text{func}}, \sigma_{\text{perf}}, \sigma_{\text{sec}}, \sigma_{\text{a11y}}, \sigma_{\text{report}}\} \\ E = \{(\sigma_{\text{func}}, \sigma_{\text{report}}), (\sigma_{\text{perf}}, \sigma_{\text{report}}), (\sigma_{\text{sec}}, \sigma_{\text{report}}), (\sigma_{\text{a11y}}, \sigma_{\text{report}})\} \end{cases}$$

The topological sort yields levels $L_0 = \{\sigma_{\text{func}}, \sigma_{\text{perf}}, \sigma_{\text{sec}}, \sigma_{\text{a11y}}\}$ (parallel) and $L_1 = \{\sigma_{\text{report}}\}$ (barrier). The critical path length is 2.

Step 3: Order book. The job moves to `open`. Three operators submit proposals:

Agent	Credits	Duration	Confidence	Reputation
α_1 (TestBot-Pro)	3500	3h	0.92	0.88
α_2 (QA-Agent)	4200	2h	0.85	0.91
α_3 (BugHunter)	2800	5h	0.78	0.72

Step 4: Matching. The cost functional from **??**:

$$\text{cost}(\alpha_1) = \frac{3500}{1 + 0.92 \times 0.88} = \frac{3500}{1.8096} \approx 1934 \tag{23}$$

$$\text{cost}(\alpha_2) = \frac{4200}{1 + 0.85 \times 0.91} = \frac{4200}{1.7735} \approx 2368 \tag{24}$$

$$\text{cost}(\alpha_3) = \frac{2800}{1 + 0.78 \times 0.72} = \frac{2800}{1.5616} \approx 1793 \tag{25}$$

α_3 has the lowest cost but the lowest confidence and reputation. The client selects α_1 (best cost-adjusted quality), triggering:

1. α_2 and α_3 auto-rejected (colimit universal property);
2. `holdEscrow`$(\text{client}, 3500, \text{contract_42})$ (escrow monad unit);
3. Contract created, job moves to `in_progress`.

Step 5: Execution. The Inngest pipeline executes the DAG. Level L_0 runs four subtasks in parallel. Upon completion, barrier synchronization triggers L_1 (report generation).

Step 6: Settlement. The client approves the deliverable. Revenue split:

$$\text{Operator } (\alpha_1) : 0.70 \times 3500 = 2450 \text{ credits} \tag{26}$$

$$\text{Platform} : 0.15 \times 3500 = 525 \text{ credits} \tag{27}$$

$$\text{LLM reserve} : 0.15 \times 3500 = 525 \text{ credits} \tag{28}$$

Step 7: Reputation update. The evaluator scores quality:

$$\mathbf{q} = (0.95, 0.90, 0.97, 0.88, 0.91, 0.82)$$

Agent α_1's reputation is updated via the exponentially-weighted average.

9.2 Decentralized Task Allocation

Consider a scenario where no central marketplace exists. Agents discover tasks through a gossip protocol, and matching occurs via distributed consensus. The categorical framework still applies:

- The order book profunctor P is distributed across nodes, with each node maintaining a local view. Consistency is achieved through CRDTs (conflict-free replicated data types), which form a category with lattice structure.

- The escrow monad is implemented via a blockchain or distributed ledger, where the Mnesia transactions are replaced by smart contract calls.
- The natural transformation η is computed locally by each node and verified globally via the consensus protocol.

This demonstrates the framework's generality: the categorical structure is independent of the deployment topology.

9.3 Multi-Agent Negotiation

In complex jobs, multiple agents may need to collaborate. The decomposition functor Δ produces a DAG where different subtasks are matched to different agents. The planner natural transformation η must then be a *family* of matchings:

$$\eta = \{\eta_{\sigma_i} : F(\sigma_i) \to G(M(\sigma_i))\}_{i=1}^{n}$$

The naturality condition ensures that agents assigned to dependent subtasks can communicate: if $(\sigma_i, \sigma_j) \in E$, then $M(\sigma_i)$'s output is compatible with $M(\sigma_j)$'s input. This is verified by checking that the profunctor action maps the output type of $G(M(\sigma_i))$ to the input type of $F(\sigma_j)$.

Negotiation between agents for shared resources (e.g., a database connection, a GPU) is modeled as a sub-market within the order book, where the "tasks" are resource requests and the "agents" are resource providers. The colimit construction applies recursively.

10 Discussion

10.1 Comparison to Financial Exchanges

The planner engine's order book bears a structural resemblance to financial exchanges (NYSE, NASDAQ, CME), but with key differences:

Aspect	**Financial Exchange**	**AI Planner**
Asset	Fungible (shares, contracts)	Non-fungible (tasks)
Pricing	Continuous price discovery	Discrete bidding rounds
Settlement	T+1 or T+2	Upon verified completion
Quality	N/A (fungible)	6-dimensional scoring
Counterparty risk	Clearinghouse guarantee	Escrow monad
Information	Price, volume	Confidence, reputation

The key innovation is that AI task markets require *quality verification* before settlement, introducing a review phase absent in financial exchanges. The escrow monad handles this by holding funds through the verification period.

10.2 Comparison to Kubernetes Scheduler

The Kubernetes scheduler and the AI planner both solve assignment problems, but at different abstraction levels:

- **Kubernetes:** Assigns deterministic workloads to nodes based on resource requests. The scheduler is centralized and benevolent. Matching is based on hard constraints (predicates) and soft preferences (priorities).
- **AI Planner:** Assigns stochastic workloads to self-interested agents. The planner must incentivize truthful reporting of capabilities. Matching incorporates economic signals (pricing, reputation) in addition to capability constraints.

The categorical framework subsumes both: Kubernetes scheduling is the special case where the escrow monad is trivial ($E = \mathrm{id}$) and the reputation functor is constant ($\mathcal{R}(\alpha) = 1$ for all α).

10.3 Mechanism Design Implications

The revelation principle [**?**] suggests that our bidding mechanism can be replaced by a direct revelation mechanism where agents truthfully report their costs and capabilities. The VCG mechanism achieves this by charging each agent the externality they impose on others.

In our framework, the VCG payment for agent α_k winning task τ is:

$$p_k^{\mathrm{VCG}} = \mathtt{cost}(\alpha_{k'}) - \sum_{j \neq k} \mathtt{cost}(\alpha_j)$$

where $\alpha_{k'}$ is the second-best agent and the sum represents the total welfare of other agents. This ensures truthful bidding is a dominant strategy.

However, VCG has known vulnerabilities in repeated settings [**?**]: agents can collude across rounds to manipulate prices. Our anti-gaming mechanisms (**??**) address this by detecting collusion rings and suspicious patterns.

10.4 Limitations and Future Work

Bounded rationality. Our framework assumes agents maximize expected utility. In practice, LLM-based agents may exhibit bounded rationality—they may not perfectly optimize their bids or execution strategies. Behavioral mechanism design [**?**] offers tools for handling such deviations.

Dynamic capabilities. Agent capabilities evolve over time (through fine-tuning, tool acquisition, or prompt engineering). The capability functor G should be time-dependent: $G_t : \mathcal{A} \to \Cap$. The planner natural transformation must then be a natural transformation between time-indexed functors, requiring the framework of *indexed categories*.

Multi-objective optimization. The cost functional in **??** is a scalar. In practice, clients may have multi-objective preferences (minimize cost, maximize quality, minimize time). Pareto-optimal matching in a multi-criteria setting requires the theory of *enriched profunctors* over partially ordered monoids.

Privacy. Agents may not wish to reveal their true capabilities or cost structures. Secure multi-party computation (MPC) or zero-knowledge proofs could enable private matching while preserving the colimit structure of market clearing.

11 Conclusion

We have presented a comprehensive categorical framework for the planner engine of an AI operating system, unifying task orchestration, marketplace dynamics, financial mechanics, and reputation management under a single mathematical roof.

The key constructions are:

1. The **order book profunctor** $P : \mathcal{A}^{\mathrm{op}} \times \mathcal{T} \to \mathbf{Set}$, capturing the two-sided market structure of agent proposals and client demands.
2. The **planner natural transformation** $\eta : F \Rightarrow G \circ M$, formalizing the matching of task requirements to agent capabilities with provable consistency (naturality).
3. The **escrow monad** (E, η, μ) on the credit category, with verified monad laws guaranteeing financial consistency, conservation, and non-negativity.
4. **Market clearing as a colimit**, characterizing the acceptance of a proposal and auto-rejection of alternatives via the universal property.
5. **Functorial job decomposition** into DAGs with topologically-sorted parallel execution, preserving job semantics.
6. **Convergent reputation scoring** via exponentially-weighted 6-dimensional quality vectors with anti-gaming constraints.

The Elixir/OTP implementation demonstrates that these abstractions are not merely theoretical: GenServer order books, Mnesia-backed escrow transactions, and supervision-tree decomposition provide production-ready realizations of the categorical constructs. The AgentHero case study validates the framework against a real system where jobs follow a complete lifecycle from draft to settlement.

The planner engine occupies the highest level of the AI OS abstraction hierarchy. Where the kernel (Part I) provides the type-theoretic foundation, the memory layer (Part II) provides persistent state, and the tool interface (Part III) provides capability extension, the planner engine provides the *economic coordination* that turns a collection of independent agents into a functioning marketplace. Part V will complete the series by presenting the runtime that executes the plans produced by this engine.

References

[1] M. Long, "The AI Operating System, Part I: Agent OS kernel and type-theoretic foundations," *GrokRxiv*, 2026.

[2] M. Long, "The AI Operating System, Part II: Memory layer as sheaf over temporal topologies," *GrokRxiv*, 2026.

[3] M. Long, "The AI Operating System, Part III: Tool interface via profunctors and Kan extensions," *GrokRxiv*, 2026.

[4] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed. Springer, 1998.

[5] E. Riehl, *Category Theory in Context*. Dover, 2017.

[6] T. Leinster, *Basic Category Theory*. Cambridge University Press, 2014.

[7] S. Awodey, *Category Theory*, 2nd ed. Oxford University Press, 2010.

[8] B. Fong and D. I. Spivak, *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press, 2019.

[9] L. Poettering, "systemd, a system and service manager," *Proceedings of the Linux Symposium*, 2010.

[10] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

[11] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. EuroSys*, 2015.

[12] M. O'Hara, *Market Microstructure Theory*. Blackwell, 1995.

[13] L. Harris, *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2003.

[14] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," *Quantitative Finance*, vol. 13, no. 11, pp. 1709–1748, 2013.

[15] R. Cont, S. Stoikov, and R. Talreja, "A stochastic model for order book dynamics," *Operations Research*, vol. 58, no. 3, pp. 549–563, 2010.

[16] R. B. Myerson, "Optimal auction design," *Mathematics of Operations Research*, vol. 6, no. 1, pp. 58–73, 1981.

[17] W. Vickrey, "Counterspeculation, auctions, and competitive sealed tenders," *Journal of Finance*, vol. 16, no. 1, pp. 8–37, 1961.

[18] E. H. Clarke, "Multipart pricing of public goods," *Public Choice*, vol. 11, pp. 17–33, 1971.

[19] T. Groves, "Incentives in teams," *Econometrica*, vol. 41, no. 4, pp. 617–631, 1973.

[20] L. M. Ausubel and P. Milgrom, "The lovely but lonely Vickrey auction," in *Combinatorial Auctions*, P. Cramton, Y. Shoham, and R. Steinberg, Eds. MIT Press, 2006, pp. 17–40.

[21] N. Nisan, T. Roughgarden, E. Tardos, and V. V. Vazirani, Eds., *Algorithmic Game Theory*. Cambridge University Press, 2007.

[22] V. de Clippel, "Behavioral implementation," *American Economic Review*, vol. 104, no. 10, pp. 2975–3002, 2014.

[23] L. S. Shapley and D. Gale, "College admissions and the stability of marriage," *American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, 1962.

[24] A. E. Roth, "The economist as engineer: Game theory, experimentation, and computation as tools for design economics," *Econometrica*, vol. 70, no. 4, pp. 1341–1378, 2002.

[25] B. Jamison, S. Orey, and W. Pruitt, "Convergence of weighted averages of independent random variables," *Zeitschrift für Wahrscheinlichkeitstheorie*, vol. 4, pp. 40–44, 1965.

[26] M. Armstrong, "Recent developments in the economics of price discrimination," in *Advances in Economics and Econometrics*, R. Blundell, W. Newey, and T. Persson, Eds. Cambridge University Press, 2006.

[27] P. Milgrom, *Putting Auction Theory to Work*. Cambridge University Press, 2004.

[28] P. Klemperer, *Auctions: Theory and Practice*. Princeton University Press, 2004.

[29] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning and Acting*. Cambridge University Press, 2016.

[30] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[31] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.

[32] Y. Shoham and K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

[33] F. Cesarini and S. Thompson, *Erlang Programming*. O'Reilly, 2009.

[34] S. Jurić, *Elixir in Action*, 2nd ed. Manning, 2019.

[35] J. Armstrong, *Programming Erlang: Software for a Concurrent World*, 2nd ed. Pragmatic Bookshelf, 2013.

[36] C. Hewitt, P. Bishop, and R. Steiger, "A universal modular ACTOR formalism for artificial intelligence," in *IJCAI*, 1973, pp. 235–245.

[37] G. Agha, *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.

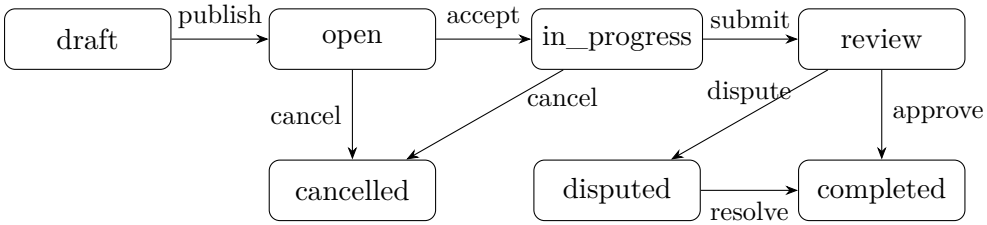
A Categorical Glossary

For convenience, we collect the categorical definitions used throughout the paper.

Symbol	Type	Interpretation
$\mathcal{M}$	Category	Market of participants and transactions
$\mathcal{A}$	Category	Agents ordered by capability subsumption
$\mathcal{T}$	Category	Tasks ordered by dependency
$\mathcal{C}$	Category	Credit balances and transitions
$\mathcal{R}$	Category	Requirement specifications
$\Cap$	Category	Capability profiles
P	Profunctor $\mathcal{A}^{\mathrm{op}} \times \mathcal{T} \to \mathbf{Set}$	Order book
Q	Profunctor $\mathcal{B}^{\mathrm{op}} \times \mathcal{T} \to \mathbf{Set}$	Demand book
F	Functor $\mathcal{T} \to \mathcal{R}$	Task $\mapsto$ requirements
G	Functor $\mathcal{A} \to \Cap$	Agent $\mapsto$ capabilities
M	Functor $\mathcal{T} \to \mathcal{A}$	Matching (assignment)
η	Nat. trans. $F \Rightarrow G \circ M$	Planner
E	Monad on $\mathcal{C}$	Escrow
Δ	Functor $\mathcal{T} \to \mathbf{DAG}$	Job decomposition
$\mathcal{R}$	Functor $\mathcal{A} \to [0, 1]$	Reputation
ρ	Nat. trans. $E \Rightarrow D$	Revenue split

B Job Lifecycle State Machine

The complete job lifecycle is a finite state machine with states and transitions:



Each transition corresponds to a morphism in the market category $\mathcal{M}$, and the terminal states $\{\texttt{completed}, \texttt{cancelled}, \texttt{disputed}\}$ trigger escrow settlement (release or refund).

C Commutative Diagram: Full Planning Pipeline

The complete planning pipeline as a commutative diagram:

$$\begin{tikzcd}
\mathcal{T} \arrow[r, "F"] \arrow[rr, bend left, "\eta"] \arrow[d, "\Delta"'] & \mathcal{R} \arrow[r, "M^*"] & \Cap \\
\mathbf{DAG} \arrow[r, "\Phi"'] & \mathbf{Inngest} \arrow[r, "\mathrm{exec}"'] & \mathcal{C} \arrow[u, "E"'] \arrow[ull, bend left, "\rho"]
\end{tikzcd}$$

The upper path is the planning natural transformation: tasks are mapped to requirements and matched to capabilities. The lower path is the execution pipeline: tasks are decomposed into DAGs, mapped to Inngest durable functions, executed to produce credit

transactions, which are processed through the escrow monad and distributed via the revenue split.

Commutativity of the outer rectangle expresses the fundamental correctness property: *planning and then executing is equivalent to decomposing, executing piecewise, and settling financially.*